

Ain Shams University

Faculty of Computer and Information Sciences

Resala Charity Management System

Software Requirements Specification

Course: CSE336 — Software Design Patterns

Instructor: Dr. Raghda Ali

Version: 1.0

Student	ID
Sharqawy Wael	23P0360
Zyad El-Nemr	23P0013
Islam Nashaat	23P0077
Mohammed Osama	23P0374
Youssef Tarek	23P0177
Fady Joseph	23P0124

4 November 2026

Contents

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions . . . 1

2 Overall description **1**
2.1 Users . . . 1
2.2 Operating environment . . . 1
2.3 Constraints . . . 2
2.4 Assumptions . . . 2
2.5 Architecture (MVC) . . . 2

3 Functional requirements **2**
3.1 Donation management . . . 3
3.2 Volunteer management . . . 3
3.3 Event management . . . 3
3.4 Beneficiary management . . . 4
3.5 Communication . . . 4
3.6 Reference data (lookup tables) . . . 4
3.7 User interface . . . 4

4 Non-functional requirements **5**

5 Main data **5**

6 Design pattern requirements (Phase 1) **6**

7 Client requirements **6**

A CRUD lookup table: event types **7**

B CRUD table: donation module (extra example) **7**

C Limits and out of scope for Phase 1 **7**

1 Introduction

1.1 Purpose

This document defines the requirements for the Resala Charity Management System. It states what the system must do, who uses it, and what is outside the scope of Phase 1.

1.2 Scope

The system helps a non-profit organization run its daily work in one application:

- **Donations:** record donors and donations, issue receipts, take payments.
- **Volunteers:** keep volunteer records, skills and hours, and issue certificates.
- **Events:** create events, register attendees, track attendance.
- **Beneficiaries:** keep beneficiary records and track the help they receive.
- **Communication:** send notifications to donors, volunteers and attendees.
- **Reference data:** keep editable lists of event types and skills.

The goal is a system that is easy to change (flexible), easy to read (maintainable), and built from reusable parts (reusable). Design patterns are used to reach these goals.

Phase 1 scope

The current version has the main management screens, local storage, reference-data maintenance, an MVC structure, and the required patterns: Strategy, Observer, Decorator, Factory and Singleton. It has a graphical interface and a console interface that share the same controllers and services.

1.3 Definitions

Term	Meaning
Donor	A person or organization that gives money or goods.
Donation	One gift: online, by check, or in kind (goods).
Beneficiary	A person or family that receives help.
Distribution	One record of help given to a beneficiary.
Staff user	The person at the branch who operates the system.
CRUD	Create, Read, Update, Delete.
MVC	Model, View, Controller: the layered structure of the code.

2 Overall description

2.1 Users

User	What they need from the system
Branch administrator	Runs the branch. Needs the full set of management screens, including reference data.
Event organiser	Runs events. Needs the event, registration and attendance screens.
Volunteer	Does not operate the system. Their record, skills and hours are managed by staff.
Donors, beneficiaries, attendees	Do not use the system. They appear as records and receive messages (email, SMS, social).

Phase 1 has no login, user accounts, or permission checks. This table describes who the system is for, not what it enforces: anyone who runs the application can open every screen.

2.2 Operating environment

- a Java application with a JavaFX graphical interface and a console interface;

- a local SQLite database;
- a normal desktop running Windows, Linux or macOS.

2.3 Constraints

- the code follows MVC: views show data, controllers take input, services hold business rules, and repositories hold the data-access SQL, while the shared database manager only opens the connection and creates the schema;
- Phase 1 uses only the six patterns taught so far (see Section 6); more patterns are added in Phase 2;
- money is stored as whole minor units (piastres) to avoid rounding errors.

2.4 Assumptions

- messages (email, SMS, social) are simulated in Phase 1, so no real provider is required;
- online payments use a fake payment provider;
- one organization uses the system on one machine;
- authentication (login) is not part of Phase 1;
- need types are supported by the beneficiary module but are not yet editable on the lookup screen.

2.5 Architecture (MVC)

The system is built in layers. The graphical and the console interface use the same controllers and services, so each business rule is written once and behaves the same in both.

Layer	Responsibility
Views	Display data, collect input and show feedback.
Controllers	Receive requests from the screens and pass them to the services.
Services	Validate data and apply the rules of each module.
Repositories	Define the storage operations without exposing storage details.
Storage implementations	Carry out the repository operations on the local database.
Core services	Shared configuration, database access, transactions and audit log.

Table 1 lists the MVC roles and what each one must not do.

MVC role	Example	Must not
Model	Donation, Volunteer, Event	Contain storage or interface code.
View	Donation screen, console donation menu	Contain business rules or open a database connection.
Controller	Donation controller	Print to the console or read the keyboard directly.
Service	Donation service	Know which screen called it.
Repository	Donation repository	Expose storage details to the service.

Table 1: MVC roles and their boundaries.

3 Functional requirements

Each requirement has an identifier, a statement and a priority: **M** = must have, **S** = should have.

3.1 Donation management

ID	Requirement	Priority
FR-D1	The system shall create, view, search, update and archive donors (name, type, email, phone).	M
FR-D2	The system shall record a donation linked to a donor, with date, amount and reference number.	M
FR-D3	The system shall support three donation types and validate each one: online needs a positive amount, check needs a check number (stored as the donation reference number), and in-kind needs an item and a quantity, with an optional estimated value.	M
FR-D4	The system shall let staff view, filter, update and delete donations.	M
FR-D5	The system shall generate a receipt for any donation. Receipts may include a thank-you note and a tax note.	M
FR-D6	The system shall generate a yearly tax statement for a donor that adds up the monetary donations of one year.	S
FR-D7	The system shall let staff take a payment by cash, card or wallet and record whether it succeeded.	S
FR-D8	The system shall add the donation to the activity feed and notify the donor with a thank-you message after a donation is recorded.	M

3.2 Volunteer management

ID	Requirement	Priority
FR-V1	The system shall create, view, update and delete volunteers (name, email, phone, availability).	M
FR-V2	The system shall manage a list of skills and attach skills to volunteers.	M
FR-V3	The system shall search volunteers by skill (available in the console interface in Phase 1).	S
FR-V4	The system shall check the skill and availability of the volunteer before assigning the volunteer to an event for a chosen skill.	M
FR-V5	The system shall track volunteer hours per event and show a summary.	M
FR-V6	The system shall generate a volunteer certificate once the volunteer reaches the required hours (20).	S

3.3 Event management

ID	Requirement	Priority
FR-E1	The system shall create, view, update and delete events (title, type, date, capacity, hours, description).	M
FR-E2	The system shall manage event types (create, rename, delete).	M
FR-E3	The system shall open and close events. Registration is allowed only while an event is open.	M
FR-E4	The system shall register attendees, issue a ticket reference, and stop at the event capacity.	M
FR-E5	The system shall check attendees in and show attendance for an event.	M
FR-E6	The system shall notify registered attendees automatically when an event is updated, opened, closed or rescheduled.	M
FR-E7	The system shall let staff send a reminder to the registered attendees of an event.	S

3.4 Beneficiary management

ID	Requirement	Priority
FR-B1	The system shall create, view, update and delete beneficiaries (name, phone, household size, need type, status).	M
FR-B2	The system shall let staff choose a need type for each beneficiary. Need types are supported as reference data, but editing them from the Lookups screen is not part of Phase 1.	M
FR-B3	The system shall record each distribution of items or services to a beneficiary, optionally linked to a donation.	M
FR-B4	The system shall show the distribution history of a beneficiary.	M
FR-B5	The system shall let staff choose how limited resources are shared: equally, by need, or first come first served.	S
FR-B6	The system shall calculate an impact score per beneficiary to help evaluate programs.	S

3.5 Communication

ID	Requirement	Priority
FR-C1	The system shall send messages by email, SMS or social media post. Delivery is simulated in Phase 1.	M
FR-C2	Staff shall be able to switch the channel at runtime from the Messages screen, without changing the rest of the system.	M
FR-C3	The system shall keep an audit log of the messages sent during the current session.	S

3.6 Reference data (lookup tables)

ID	Requirement	Priority
FR-R1	The system shall keep event types and skills as editable lists with full create, read, update and delete.	M
FR-R2	A name shall be required, trimmed and unique.	M
FR-R3	Renaming an item shall update every event or volunteer that uses it.	M
FR-R4	Deleting an item that is already used by an event or a volunteer shall be refused, to protect existing records.	M
FR-R5	Every change shall refresh the open screens, so the event form shows the new list at once.	M

The CRUD table for event types is in Appendix A.

3.7 User interface

ID	Requirement	Priority
FR-U1	The system shall provide a graphical interface (JavaFX) for all main screens.	M
FR-U2	The system shall also provide a console interface that offers the same operations through the same controllers and services, so no rule is written twice.	S
FR-U3	The donation, volunteer, event, beneficiary and reference-data modules shall offer CRUD for their main records from the interface.	M

4 Non-functional requirements

ID	Category	Requirement	Priority
NFR-1	Flexibility	A new receipt add-on or distribution rule shall be added by adding a new class, without editing existing classes. A new donation type or message channel requires one new class plus a small registration change.	M
NFR-2	Maintainability	Code shall be split into MVC layers. Controllers contain no SQL. Data access goes through repositories; the shared database manager only opens connections and creates the schema.	M
NFR-3	Reusability	Shared parts (database access, configuration, notification) shall be implemented once and reused by all modules.	M
NFR-4	Usability	A staff user shall complete common tasks (add donor, record donation, register attendee) in a few steps without training.	M
NFR-5	Reliability	Failed operations shall show a clear message and shall not leave half-saved data. Payments shall use a transaction.	M
NFR-6	Performance	Screens shall respond in under 2 seconds for normal data sizes.	S
NFR-7	Data integrity	Donors are archived, not deleted, so donation history stays valid.	M
NFR-8	Portability	The application shall run on any system with Java installed. No separate database server is needed.	S
NFR-9	Consistency	The graphical and the console interface shall give the same result for the same action, because they share controllers and services.	M
NFR-10	Usability	Open screens shall refresh after a change, so users never see an outdated list.	M

5 Main data

Entity	Key fields
Donor	id, name, type, email, phone, active
Donation	id, donor, type, date, reference number, status (plus amount, check number for checks, or item details for in-kind)
Payment	id, payer, amount, fee, currency, method, status
Volunteer	id, name, email, phone, available
Skill	id, name
Event	id, title, event type, date, capacity, hours, status
EventRegistration	id, event, attendee name and email, ticket reference, checked in
VolunteerEventAssignment	id, volunteer, event, skill, work date, hours
Beneficiary	id, name, phone, household size, need type, status
Distribution	id, beneficiary, donation (optional), item, quantity, date
EventType	id, name (editable lookup for events)

The main relationships are: a donor makes many donations; an event has many registrations; a volunteer can be assigned to many events; a beneficiary receives many distributions.

6 Design pattern requirements (Phase 1)

Pattern	Where it is required	Why
Singleton	Database connection, configuration, audit log.	One shared instance for the whole application.
Factory	Creating online, check and in-kind donations, each with its own validation; also creating the family of console screens.	Add a donation type without changing the caller.
Strategy	Message channels (email, SMS, social); distribution policies (equal, by need, first come first served).	Swap behavior at runtime.
Observer	After a donation: activity-feed entry and thank-you message. When an event is updated, opened, closed or reminded: inform the registered attendees.	Services do not need to know who reacts.
Decorator	Receipt with optional thank-you and tax additions.	Combine add-ons freely; two options would otherwise need four receipt subclasses.
MVC	Whole application: two complete interfaces (graphical and console) share the same controllers and services.	Separate screens, input handling and business rules.

The class diagram in the Phase 1 report shows all main classes and where each pattern is applied.

7 Client requirements

The requirements below describe the problems the branch staff face and the outcome the client needs, not just the screens that were built. Each row states the need and the requirement that delivers it.

Table 2: Client needs and their implementation.

ID	Client need and expected outcome	Addressed by
CR1	**Year-end statements.** At year end, staff rebuild each donor's total by hand. The client needs an accurate statement for any donor and year, covering monetary donations only, without re-entering data.	**FR-D6**; Donations → Yearly Statement
CR2	**Safe volunteer assignment.** Coordinators assign volunteers from memory, which causes skill mismatches and no-shows. The client needs every assignment checked against availability and the required skill before it is confirmed.	**FR-V4**; Volunteers → Register to Event
CR3	**Volunteer certificates.** Certificates are calculated by hand and issued late. The client needs a certificate produced only after 20 verified hours, showing the correct total.	**FR-V5, FR-V6**; Volunteers → Print Certificate
CR4	**Event changes.** Attendees are not told when an event changes, so some travel for nothing. The client needs every registered attendee informed automatically when the event is updated, opened, closed or rescheduled.	**FR-E6**; automatic, and Events → Send Reminder
CR5	**Aid history.** Aid is recorded on paper, so the branch cannot say who received what, when, or whether it was already given. The client needs one history per beneficiary that staff can check before distributing again.	**FR-B3, FR-B4**; Beneficiaries → Distribute Aid / History
CR6	**Staff-maintained lists.** Every new event type or skill currently needs a developer. The client needs staff to add, rename and remove these values themselves without breaking existing records.	**FR-R1 to FR-R5**; Lookups
CR7	**One message screen.** Supporters use different channels, and the branch cannot run a separate tool for each. The client needs one message screen where the channel can be changed immediately.	**FR-C1 to FR-C3**; Messages

A CRUD lookup table: event types

Event type is the Phase 1 lookup-table example. It is an editable value used when creating an event. C = Create, R = Read, U = Update, D = Delete.

Function / use case	EventType	Event	Interface entry point
Create a new event type	C	–	Lookups → Add
Browse all event types	R	–	Lookups table; event form
Rename an event type	U	–	Lookups → Rename
Delete an unused event type	D	R (guard)	Lookups → Delete
Assign a type when creating or editing an event	R	C / U	Events → Create / Edit

Skills use the same screen and the same four operations. A skill is used by the volunteer–skill relation, so the delete guard protects volunteer records in the same way.

B CRUD table: donation module (extra example)

C = Create, R = Read, U = Update, D = Delete. For donors, "D" means archive.

Function / requirement	Donor	Donation	Payment
Register a donor	C	–	–
Search donors	R	–	–
Edit donor details	U	–	–
Archive donor	D	–	–
Record a donation	R	C	–
View and filter donations	R	R	–
Edit a donation	–	U	–
Delete a donation	–	D	–
Take an online payment	R	C	C
Generate a receipt	R	R	–
Generate a yearly tax statement	R	R	–
View payment history	R	–	R

C Limits and out of scope for Phase 1

The following items are planned for Phase 2 and are not required now: admin pages, the extra patterns (Command, Adapter, Facade, Template Method, Iterator, State, Proxy), and real email or SMS providers.

Current limits of Phase 1:

- payment and messaging are simulated;
- authentication is not part of this phase, so there are no user accounts or permission checks; anyone who runs the application can use every screen;
- need types are supported by the beneficiary module but are not yet exposed on the lookup screen.